DAA Assignment 1

Design and Analysis of Algorithms

ASSIGNMENT 1

CO1: *Algorithm fundamentals, efficiency analysis, asymptotic analysis, mathematical analysis, recurrence analysis, Master Theorem, substitution, etc.*

Assignment Title: Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System (Total Marks - 50)

Duration:1week

Scenario — University Student Record Retrieval

A university maintains a large academic database containing student records, each identified by a unique Registration Number. Faculty and administrative staff frequently need to determine whether a particular student record exists and retrieve the corresponding information.

The database may contain:

1,000 → 10,000 → 100,000 → 1,000,000+ records

You are required to investigate, design and compare efficient algorithmic approaches for locating a given registration number as the database grows.

Your solution must remain practical for both small and very large datasets. Any preprocessing performed on the records must be included in your efficiency analysis.

Design Challenge:

The university now needs to process 50,000 search requests per hour on a database containing 1,000,000 student records. New student records are added only once per day.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Design the Algorithm **5 Marks**

Design and propose **at least three meaningfully different algorithmic strategies** for locating a given registration number.

The three approaches must represent **distinct algorithmic strategies**. Iterative and recursive implementations of the **same underlying algorithm will not be considered as two different approaches**.

Your proposed approaches must include:

- At least **one iterative approach**
- At least **one recursive approach**
- A clear explanation of the basic idea behind each approach
- Pseudocode or an appropriate flowchart for each approach
- A brief justification for selecting each approach for investigation
- Any preprocessing required by the approach, such as sorting or restructuring the data

You are expected to **design and select the approaches independently**. The appropriateness and diversity of the selected algorithmic strategies form part of the assessment.

Evidence Expected

Three meaningfully different algorithmic strategies, supported by pseudocode/flowcharts, identification of the design strategy, preprocessing requirements, and justification for selecting each approach.

TASK 2 – Identify the Basic Operation 2 Marks

For each of the proposed algorithms, identify the **basic operation** that predominantly determines its execution time.

Discuss:

- What the basic operation is
- Where it occurs in the algorithm
- Why it is appropriate to use this operation for efficiency analysis
- How the number of times it executes may change as the input size (n) increases

Do not simply name an operation. You are expected to **relate the selected operation to the behaviour of the algorithm**.

Evidence Expected

Correct identification and justification of the dominant/basic operation for each proposed algorithm.

TASK 3 – Derive the Time-Complexity Function Mathematically 6 Marks

Derive the time-complexity function for each proposed approach using mathematical analysis.

Discuss:

- The input size (n)
- The number of times the basic operation executes
- How the operation count changes with (n)
- The mathematical expression representing the running time
- Any preprocessing cost involved
- The overall cost when preprocessing and searching are considered together

You must show the **mathematical reasoning leading to the complexity result**.

Merely stating that an algorithm is (O(n)), (O(\log n)), etc., will not receive full credit.

Evidence Expected

A clear mathematical derivation of the time-complexity function for each approach, including relevant operation counts and preprocessing costs.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour 4 Marks

Analyse the performance of each algorithm under different input conditions.

Discuss, wherever applicable:

- What input condition produces the **best case**
- What input condition produces the **average case**
- What input condition produces the **worst case**
- How the number of operations changes in each case
- Why these differences occur

Do not simply provide a table containing complexity values. You are expected to **explain the conditions that lead to each case**.

Evidence Expected

A justified best-, average- and worst-case analysis for the proposed approaches, supported by appropriate mathematical reasoning.

TASK 5 – Perform Asymptotic Analysis 3 Marks

Express and compare the growth behaviour of your proposed algorithms using appropriate asymptotic notation.

Discuss:

- Appropriate use of (O), (\Omega) and/or (\Theta), wherever relevant
- How each algorithm behaves as (n) becomes very large
- How the growth rates of the proposed approaches differ
- What the growth behaviour means when the dataset increases from **1,000 to 1,000,000+ records**

You are expected to **interpret the asymptotic behaviour in the context of the given application**, rather than merely state the notation.

Evidence Expected

Correct asymptotic representation and a meaningful comparison of the growth behaviour of the proposed approaches.

TASK 6 – Formulate the Recurrence Relation 3 Marks

For the recursive approach, formulate a recurrence relation directly from the behaviour of the algorithm.

Clearly identify:

- The input size of the original problem
- The size of the recursive subproblem
- The number of recursive calls
- The work performed outside the recursive call
- The base case
- How these components lead to the recurrence relation

The recurrence must be **derived from your algorithm**, rather than simply writing a known recurrence formula.

Evidence Expected

A correctly formulated recurrence relation with clear identification of its components and base case.

TASK 7 – Solve the Recurrence Relation 4 Marks

Solve the recurrence relation obtained in Task 6 using an appropriate analytical method.

You may use:

- Substitution method
- Iteration method
- Master Theorem
- Another appropriate analytical technique

Discuss:

- Why the selected method is applicable
- The steps involved in solving the recurrence
- The resulting complexity
- What the result indicates about the behaviour of the recursive algorithm as (n) increases

You are expected to **show the analytical process**, rather than provide only the final complexity.

Evidence Expected

A complete recurrence solution using an appropriate method, together with justification of the method and interpretation of the result.

TASK 8 – Implement and Test the Algorithms 4 Marks

Implement all the proposed algorithmic approaches and experimentally test their behaviour.

Use datasets of approximately:

1,000 → 10,000 → 100,000 → 1,000,000 records

Ensure that:

- All algorithms produce the correct result
- The same or equivalent datasets are used for comparison
- Testing conditions are kept reasonably consistent
- Appropriate test cases are included
- The implementation corresponds to the algorithms analysed in the previous tasks

The purpose of implementation is **not merely to demonstrate working code**, but to generate evidence for validating your theoretical analysis.

Evidence Expected

Working implementations of the proposed algorithms together with a clearly described and reproducible testing procedure.

TASK 9 – Record Experimental Performance 3 Marks

Collect and present experimental performance data for each algorithm across the different input sizes.

Record appropriate measures such as:

- Execution time
- Number of comparisons
- Number of key operations
- Input size
- Other relevant performance measures, where justified

Present the results using appropriate:

- Tables
- Graphs
- Performance plots

Ensure that the presentation allows the different approaches to be easily compared.

Evidence Expected

Systematically collected experimental data presented clearly using appropriate tables and/or graphs.

TASK 10 – Compare Theory with Experimental Behaviour 4 Marks

Compare your theoretical analysis with your experimental results.

Discuss:

- Whether the observed growth agrees with the theoretical complexity
- Which algorithm scales better as (n) increases
- Any differences between theoretical prediction and actual execution
- Possible reasons for the differences

You are expected to **interpret the results**, rather than simply reproduce tables or graphs.

Evidence Expected

A reasoned comparison between theoretical and experimentally observed performance.

TASK 11 – Recommend and Defend the Final Design 4 Marks

Based on your mathematical analysis and experimental findings, recommend the most appropriate algorithmic solution for the university student-record system.

Discuss:

- Which approach you recommend
- Why it is more appropriate than the alternatives
- Its theoretical efficiency
- Its experimentally observed performance
- Any preprocessing cost involved
- Relevant trade-offs or limitations
- Whether the solution remains practical as the database grows

Your recommendation must be based on **evidence obtained from your own analysis and experimentation**.

Simply identifying the algorithm with the lowest Big-O complexity is not sufficient.

Evidence Expected

A technically justified final design recommendation supported by both mathematical analysis and experimental evidence.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Changed Requirement

The university now needs to process **50,000 search requests per hour** on a database containing **1,000,000 student records**.

However, new student records are added only **once per day**.

Re-evaluate your original design in response to these changed operating conditions.

You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

Discuss:

- What has changed from the original problem
- Which original assumptions are affected
- Why the frequency of searches is now important
- Why the relatively low frequency of record updates may influence the design

Evidence Expected

A clear analysis of how the changed requirements affect the original algorithmic design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your original recommended solution remains appropriate.

Discuss:

- Its expected behaviour under 50,000 searches per hour
- Whether its preprocessing cost is acceptable under the new conditions
- Whether another approach could provide better overall performance

Evidence Expected

A technically supported evaluation of the suitability of the original design under the new operating conditions.

C. Adaptation or Redesign – 2 Marks

Decide whether to:

Retain, modify or redesign the original solution.

If you modify or redesign it:

- Describe the revised algorithmic strategy
- Explain what has changed
- Determine the effect on computational complexity

If you retain the original solution, provide sufficient technical justification for why no redesign is necessary.

Evidence Expected

An appropriate design response to the changed requirement, with clear algorithmic reasoning.

D. Evidence-Based Defence – 2 Marks

Defend your final design decision using evidence.

Use:

- Mathematical complexity analysis
- Experimental results, where appropriate
- Search frequency
- Preprocessing cost
- Dataset size
- Relevant computational trade-offs

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final design decision integrating theoretical analysis, experimental evidence and the changed system constraints.

MARK DISTRIBUTION

Task	Marks
Task 1 – Algorithm Design	5
Task 2 – Basic Operation	2
Task 3 – Mathematical Complexity Analysis	6
Task 4 – Best/Average/Worst-Case Analysis	4
Task 5 – Asymptotic Analysis	3
Task 6 – Recurrence Formulation	3
Task 7 – Recurrence Solution	4
Task 8 – Implementation & Testing	4
Task 9 – Experimental Performance	3
Task 10 – Theory vs Experiment	4
Task 11 – Final Design Recommendation	4
Subtotal	**42**
Task 12 – Day-4 Design Challenge	**8**
TOTAL	**50**

Introduction

A university maintains a large database of student records, where each record is identified by a unique Registration Number. Faculty and administrative staff need to search for student records quickly and accurately.

As the database grows from 1,000 to 1,000,000 or more records, the search algorithm must provide good performance while considering preprocessing costs, execution time, and memory usage.

This assignment investigates three different searching strategies:

1. Linear Search – an iterative sequential searching algorithm.
2. Binary Search – a recursive divide-and-conquer algorithm that requires sorted data.
3. Hash Table Search – a hash-based searching algorithm that uses a hash function to locate records.

The algorithms are analysed mathematically, implemented in Python, and compared experimentally. Finally, the design is evaluated for a system that performs 50,000 searches per hour on one million student records, with new records added once per day.

Task 1 – Design the Algorithm

1.1 Algorithm 1: Linear Search

Design strategy: Brute-force search.

Basic idea: Linear Search examines student registration numbers one by one until the required registration number is found or the end of the dataset is reached.

Preprocessing required: None. The records can be in any order.

Justification: It is simple to implement and suitable for small datasets or situations where records are unsorted.

Pseudocode

```
BUILD_HASH_TABLE(A)
   create an empty hash table H

   for each student in A
      index = HASH(student.registration_number)
      insert student into H[index]

   return H
```

```
HASH_SEARCH(H, key)
   index = HASH(key)

   for each student in H[index]
      if student.registration_number == key
         return student

   return NOT_FOUND
```

Collision handling: If two registration numbers map to the same index, both records are stored in the same bucket. The search compares registration numbers within that bucket.

Summary of the three approaches

Feature	Linear Search	Binary Search	Hash Table
Design strategy	Brute force	Divide and conquer	Hashing
Implementation	Iterative	Recursive	Iterative lookup
Requires sorted data	No	Yes	No
Preprocessing	None	Sorting	Building table
Average search time	$\Theta(n)$	$\Theta(\log n)$	Expected $\Theta(1)$
Worst-case search time	$\Theta(n)$	$\Theta(\log n)$	$\Theta(n)$
Extra storage	Small	Small, excluding sorting	$\Theta(n)$

Task 2 – Identify the Basic Operation

The basic operation is the operation performed repeatedly that predominantly determines the algorithm's execution time.

Algorithm	Basic operation	Where it occurs	Effect as n increases
Linear Search	Compare registration number with the target	Inside the loop	May require up to n comparisons
Binary Search	Compare the target with the middle registration number	At each recursive call	Search space is halved each time
Hash Table Search	Compare the target with a record in the selected bucket	During collision resolution	Expected constant number of comparisons with a suitable hash function and load factor

Explanation

Linear Search: Each record may need to be checked. If the target is absent, all n records are compared. Therefore, the comparison count increases linearly with the dataset size.

Binary Search: Each comparison eliminates approximately half the remaining search space. Consequently, the number of comparisons grows logarithmically rather than linearly.

Hash Table Search: The hash function identifies a bucket, after which registration numbers in that bucket are compared. With a good hash function and a controlled load factor, the expected bucket size remains small even when the database grows. Poor hashing or excessive collisions can increase the comparison count significantly.

Task 3 – Derive the Time-Complexity Function Mathematically

Let n represent the number of student records.

3.1 Linear Search

Suppose the target is found at position k, using one-based positions. The number of registration-number comparisons is

$$C_L(n) = k$$

If the target is absent, every record must be examined:

$$C_L(n) = n$$

For an equally likely successful search among all positions,

\[\begin{aligned} C_{\text{avg}}(n) &=\frac{1}{n}\sum_{k=1}^{n}k\\ &=\frac{n(n+1)}{2n}\\ &=\frac{n+1}{2} \end{aligned} \]

Therefore, average successful search requires approximately $n/2$ comparisons.

The overall search cost is

$$T_L(n) = aC_L(n) + b$$

where a is the cost per comparison and b represents fixed overhead.

Thus, worst-case time is $\Theta(n)$. No preprocessing is needed, so the total cost for q searches is approximately

$$T_{total} = qT_L(n)$$

3.2 Recursive Binary Search

Binary Search halves the search space at each recursive call. In the worst case, its comparison count satisfies

\[C_B(n)=C_B\left(\left\lfloor\frac n2\right\rfloor\right)+1 \]

with $C_B(1) = 1$.

For simplicity, let $n = 2^m$. Repeated expansion gives

\[\begin{aligned} C_B(n)&=C_B(n/2)+1\\ &=C_B(n/4)+2\\ &=C_B(n/8)+3\\ &=\cdots\\ &=C_B(1)+m \end{aligned} \]

Since $m = \log_2 n$,

$$C_B(n) = 1 + \log_2 n$$

up to rounding and the precise base-case convention. Hence, binary search requires $\Theta(\log n)$ comparisons in the worst case.

However, the array must first be sorted. Using an efficient comparison sort such as merge sort, preprocessing takes

$$T_{sort}(n) = \Theta(n \log n)$$

For q searches on the sorted dataset,

$$T_{total} = \Theta(n \log n) + \Theta(q \log n)$$

Sorting is performed once when the dataset is prepared and can be reused for subsequent searches.

3.3 Hash Table Search

Building a hash table involves inserting all n records.

With a suitable hash function and controlled load factor, the expected cost per insertion is constant. Therefore,

$$T_{build}(n) = \Theta(n)$$

For an individual lookup, the expected number of records examined in the selected bucket remains bounded under the usual uniform-hashing assumption. Thus,

$$E[T_H(n)] = \Theta(1)$$

in the expected case.

If all keys collide into one bucket, however, a lookup may examine all n records:

$$T_{H,worst}(n) = \Theta(n)$$

For q searches after building the table,

$$T_{total} = \Theta(n) + \Theta(q)$$

in the expected case. Therefore, the total expected cost is $\Theta(n + q)$.

Preprocessing and total-cost comparison

Algorithm	Preprocessing cost	Cost per search	Total cost for q searches
Linear Search	$\Theta(1)$	$\Theta(n)$ worst case	$\Theta(qn)$
Binary Search	$\Theta(n \log n)$	$\Theta(\log n)$	$\Theta(n \log n + q \log n)$
Hash Table	$\Theta(n)$ expected	$\Theta(1)$ expected	$\Theta(n + q)$ expected

These expressions show why preprocessing must be included: binary search is fast per query, but sorting has a cost; hashing requires construction and additional memory, but supports fast repeated lookups.

Task 4 – Analyse Best, Average and Worst-Case Behaviour

The performance of an algorithm depends on the location of the target, whether the target exists, the ordering of records, and—in hashing—the distribution of keys.

4.1 Linear Search

- Best case: The target is the first record. Only one comparison is needed, so $C_{\min}(n) = 1$.
- Average case: If the target is equally likely to occupy any position, the expected number of comparisons is $(n + 1)/2$.
- Worst case: The target is the last record or does not exist. All n records are examined, so $C_{\max}(n) = n$.

The average comparison count increases proportionally with the number of records because the algorithm does not eliminate any portion of the search space.

4.2 Recursive Binary Search

- Best case: The target matches the middle element in the first comparison. Therefore, $C_{\min}(n) = 1$.
- Average case: For a successful search with targets distributed uniformly across sorted positions, the expected number of comparisons is $\Theta(\log n)$.
- Worst case: The target is absent or is located at a position requiring the maximum number of recursive steps. The search space is halved until the target is found or the interval becomes empty.

The maximum number of comparisons is bounded by approximately \(\lfloor\log_2 n\rfloor+1\), depending on the exact implementation and boundary conditions.

4.3 Hash Table Search

- Best case: The target is the first entry checked in its bucket. The lookup takes constant time.
- Average case: With a good hash function, uniform key distribution, and controlled load factor, the expected number of bucket comparisons is constant.
- Worst case: All records collide into the same bucket, and the target is at the end or absent. Up to n records may be examined.

Hash table performance depends on hash quality, collision handling, table capacity, and load factor. Its constant-time performance is an expected-case result, not an unconditional worst-case guarantee.

Comparison of cases

Algorithm	Best case	Average case	Worst case
Linear Search	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Binary Search	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$
Hash Table Search	$\Theta(1)$	Expected $\Theta(1)$	$\Theta(n)$

Task 5 – Perform Asymptotic Analysis

Asymptotic analysis describes how the running time grows as the input size becomes large.

- Big-O, $O(f(n))$: An asymptotic upper bound.
- Big-Omega, $\Omega(f(n))$: An asymptotic lower bound.
- Big-Theta, $\Theta(f(n))$: A tight asymptotic bound.

5.1 Asymptotic complexity

Algorithm	Best-case bound	Average-case growth	Worst-case bound
Linear Search	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Binary Search	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$
Hash Table Search	$\Theta(1)$	Expected $\Theta(1)$	$\Theta(n)$

For the preprocessing phase:

- Linear Search requires no sorting.
- Binary Search requires $\Theta(n \log n)$ preprocessing when comparison-based merge sort is used.
- Hash Table construction takes expected $\Theta(n)$ time.

5.2 Effect of increasing the database size

The following table shows approximate comparison counts for a successful search. Linear Search uses its average-case count; Binary Search uses its logarithmic bound; Hash Table Search uses the ideal expected-case model.

Records (n)	Linear Search average	Binary Search upper estimate	Hash Table expected
1,000	500.5	11	Constant
10,000	5,000.5	15	Constant

100,000	50,000.5	18	Constant
1,000,000	500,000.5	21	Constant

The binary-search estimates use \(\lfloor\log_2 n\rfloor+1\) as a convenient upper estimate. Actual counts depend on the target's position.

Interpretation: Increasing the database from 1,000 to 1,000,000 records multiplies its size by 1,000. Linear Search's average comparison count also grows approximately 1,000-fold. Binary Search's comparison count increases only from around 11 to around 21. Hashing is expected to keep lookup work approximately constant if its assumptions hold.

This makes Binary Search and Hash Table Search more scalable than Linear Search for large datasets.

Task 6 – Formulate the Recurrence Relation

The recursive algorithm selected for recurrence analysis is Binary Search.

Let n be the number of records in the current search interval.

At each recursive call:

1. One middle element is selected.
2. The target is compared with that element.
3. Only one half of the interval is searched recursively.
4. The other half is discarded.

The subproblem size is approximately $n/2$. There is one recursive call and constant work outside that call.

Therefore, the recurrence relation for the worst-case running time is

\[T(n)=T\left(\left\lfloor\frac n2\right\rfloor\right)+c \]

where c is a constant representing the comparisons, midpoint calculation, and other fixed work at each call.

The base case is

$$T(1) = d, T(0) = d_0$$

where d and d_0 are constants.

For asymptotic analysis, the recurrence is commonly written as

$$T(n) = T(n/2) + \Theta(1)$$

This recurrence is derived directly from the algorithm: the input is halved, one recursive call is made, and constant work is performed at each level.

Task 7 – Solve the Recurrence Relation

We solve

$$T(n) = T(n/2) + c$$

using the iteration method.

Assume $n = 2^m$ to simplify the derivation.

Step 1: Expand the recurrence

$$T(n) = T(n/2) + c$$

Expanding again,

$$T(n) = T(n/4) + 2c$$

Expanding a third time,

$$T(n) = T(n/8) + 3c$$

After i expansions,

$$T(n) = T(n/2^i) + ic$$

Step 2: Determine the number of expansions

The recursion reaches the base case when

$$\frac{n}{2^i} = 1$$

Therefore,

$$n = 2^i$$

Taking logarithms,

$$i = \log_2 n$$

Step 3: Substitute into the expanded equation

At the base case, $T(1) = d$. Hence,

$$T(n) = d + c\log_2 n$$

Since c and d are constants, the running time is

\[\boxed{T(n)=\Theta(\log n)} \]

Step 4: Interpretation

Binary Search requires logarithmically increasing work because every recursive call eliminates half of the remaining records.

For one million records, the search interval becomes empty or reaches the target after roughly 20 levels, with the precise number of comparisons depending on the input and implementation.

The iteration method is appropriate because the recurrence contains one subproblem of half the original size and constant work per call.

Task 8 – Implement and Test the Algorithms

The following Python program implements all three strategies and tests them on datasets containing 1,000, 10,000, 100,000, and 1,000,000 registration numbers.

It records:

- Input size
- Whether the record was found
- Number of key comparisons

- Search execution time
- Preprocessing time

The program uses the same registration-number dataset for all three approaches. It measures preprocessing separately from the search operation and also reports total preparation-plus-search time for one query.

Python implementation

Copy the following program into VS Code and save it as student_search.py.

```
import time

import random


def linear_search(a, key):
    count = 0
    for i, x in enumerate(a):
        count += 1
        if x == key:
            return i, count
    return -1, count


def binary_search(a, key, low, high):
    if low > high:
        return -1, 0

    mid = (low + high) // 2

    if a[mid] == key:
        return mid, 1
```

```
    if key < a[mid]:
        pos, count = binary_search(a, key, low, mid - 1)
    else:
        pos, count = binary_search(a, key, mid + 1, high)

    return pos, count + 1

def build_hash(a):
    h = {}
    for x in a:
        h[x] = x
    return h

def hash_search(h, key):
    return h.get(key, -1), 1

random.seed(42)
sizes = [1000, 10000, 100000, 1000000]

print("N,Algorithm,Found,Comparisons,Prep_ms,Search_ms,Total_ms")

for n in sizes:
    data = list(range(1000000, 1000000 + n))
    target = data[-1]
```

```
start = time.perf_counter()
linear_search(data, target)
prep = 0
t0 = time.perf_counter()
pos, count = linear_search(data, target)
elapsed = (time.perf_counter() - t0) * 1000
print(f"{n},Linear,{pos != -1},{count},{prep:.3f},"
      f"{elapsed:.3f},{elapsed + prep:.3f}")

start = time.perf_counter()
sorted_data = sorted(data)
prep = (time.perf_counter() - start) * 1000
t0 = time.perf_counter()
pos, count = binary_search(
    sorted_data, target, 0, n - 1
)
elapsed = (time.perf_counter() - t0) * 1000
print(f"{n},Binary,{pos != -1},{count},{prep:.3f},"
      f"{elapsed:.3f},{elapsed + prep:.3f}")

start = time.perf_counter()
h = build_hash(data)
prep = (time.perf_counter() - start) * 1000
t0 = time.perf_counter()
```

```
    pos, count = hash_search(h, target)
    elapsed = (time.perf_counter() - t0) * 1000
    print(f"{n},Hash,{pos != -1},{count},{prep:.3f},"
          f"{elapsed:.3f},{elapsed + prep:.3f}")
```

Output

```
========== RESTART: C:/Users/HP/Downloa
N,Algorithm,Found,Comparisons,Prep_ms,Search_ms,To
1000,Linear,True,1000,0.000,0.134,0.134
1000,Binary,True,10,0.019,0.016,0.035
1000,Hash,True,1,0.153,0.005,0.158
10000,Linear,True,10000,0.000,1.200,1.200
10000,Binary,True,14,0.292,0.015,0.308
10000,Hash,True,1,1.033,0.004,1.037
100000,Linear,True,100000,0.000,11.830,11.830
100000,Binary,True,17,1.617,0.019,1.636
100000,Hash,True,1,21.655,0.005,21.660
1000000,Linear,True,1000000,0.000,136.341,136.341
1000000,Binary,True,20,17.320,0.021,17.340
1000000,Hash,True,1,121.326,0.006,121.332
>
```

Task 9 – Record Experimental Performance

The experimental results must be collected by running the Python program on the computer used for the assignment. Execution times depend on processor speed, available memory, Python version, and system load; therefore, the following tables provide the format for recording the actual results rather than invented measurements.

9.1 Experimental results table

Record the values printed by the program in the following table.

Dataset size	Algorithm	Preprocessing (ms)	Search (ms)	Comparisons
1,000	Linear Search	0	Record output	Record output

1,000	Binary Search	Record output	Record output	Record output
1,000	Hash Table	Record output	Record output	1 lookup
10,000	Linear Search	0	Record output	Record output
10,000	Binary Search	Record output	Record output	Record output
10,000	Hash Table	Record output	Record output	1 lookup
100,000	Linear Search	0	Record output	Record output
100,000	Binary Search	Record output	Record output	Record output
100,000	Hash Table	Record output	Record output	1 lookup
1,000,000	Linear Search	0	Record output	Record output
1,000,000	Binary Search	Record output	Record output	Record output
1,000,000	Hash Table	Record output	Record output	1 lookup

The target in the supplied program is the last registration number, making Linear Search perform n comparisons. Binary Search uses a sorted dataset, while the hash table is built before the search.

9.2 Expected performance trends

Theoretical Search Growth

Illustrative operation counts as the dataset grows, not measured execution times.

Linear SearchBinary Search

0300,000600,000900,0001,200,0001,00010,000100,0001,000,000

This graph illustrates theoretical growth for a worst-case linear search and an approximate logarithmic search bound. It is not a graph of measured execution times.

Expected observations:

- Linear Search's comparisons increase directly with dataset size.
- Binary Search requires only a small increase in comparisons as the dataset grows.
- Hash Table Search is expected to remain fast after construction, provided the hash table maintains a suitable load factor.
- Hash Table construction and Binary Search sorting require additional time before the first search.

For a more reliable graph of actual execution time, export the program output to a CSV file and plot the measured search times and total times separately.

Task 10 – Compare Theory with Experimental Behaviour

The theoretical analysis predicts the growth of each algorithm, while the experiments measure its behaviour on a real computer.

10.1 Comparison

Algorithm	Theoretical prediction	Expected experimental behaviour
Linear Search	$\Theta(n)$ worst case	Search time rises approximately in proportion to the number of records
Binary Search	$\Theta(\log n)$ search	Search time grows slowly, although sorting adds preparation cost

Hash Table	Expected $\Theta(1)$ lookup	Lookup time is usually nearly constant, while construction time increases with the number of records

10.2 Interpretation of the results

Linear Search: When the target is at the end of the dataset, every registration number must be checked. Increasing the dataset from 1,000 to 1,000,000 records increases the number of comparisons from 1,000 to 1,000,000. This supports the linear complexity prediction.

Binary Search: Each comparison reduces the search interval by approximately half. Its number of comparisons increases from around 10 to around 20 as the dataset increases from 1,000 to 1,000,000 records. However, the sorting cost must be included when the dataset is not already sorted.

Hash Table Search: The table requires time and memory to build, but subsequent lookups are expected to take constant time. The actual timing may fluctuate because of hashing overhead, memory allocation, cache behaviour, and system load.

10.3 Reasons for differences between theory and experiment

1. Hardware differences: Processor speed and memory performance affect execution time.
2. Programming-language overhead: Python's function calls and object management affect the measured results.
3. Preprocessing: Sorting and table construction may dominate the cost of a single query.
4. Input distribution: Linear Search behaves differently for a first-position target and a last-position target.
5. Hash collisions: Poor hash distribution can cause slower lookups than expected.
6. Measurement variation: Operating-system activity and timer precision can cause differences between repeated runs.

Conclusion: The theoretical analysis provides the expected growth rates. Experimental measurements should be used to verify whether those trends appear in the implementation and to determine the practical cost of preprocessing and searching.

Task 11 – Recommend and Defend the Final Design

Recommended initial design: Hash Table Search

For a large student-record database with frequent registration-number lookups, a hash table is the recommended initial design, provided that sufficient memory is available and the hash implementation is reliable.

Justification

1. Search efficiency

Linear Search requires $\Theta(n)$ time in the worst case. Binary Search requires $\Theta(\log n)$ time per query after sorting. A well-designed hash table offers expected $\Theta(1)$ lookup time.

2. Preprocessing cost

Building the hash table takes expected $\Theta(n)$ time. This is less than the $\Theta(n \log n)$ cost of comparison-based sorting for Binary Search. Once the table is built, it can be reused for many queries.

3. Practical scalability

For one million records, Linear Search may require one million comparisons for a missing target. Binary Search requires about 20 comparisons in a typical worst-case bound. Hash Table Search is expected to inspect only a small number of records in the relevant bucket.

4. Limitations

Hash tables use additional memory and can suffer from collisions. They also do not maintain records in sorted order, so a separate index or sorting mechanism may be needed for range queries, alphabetical listings, or ordered reports.

Final comparison

Criterion	Linear Search	Binary Search	Hash Table
Search speed for repeated lookups	Low	High	Very high on average
Preprocessing	None	Sorting required	Table construction
Memory overhead	Low	Low beyond sorted storage	Higher

Ordered/range queries	No efficient support	Good	Not directly supported
Recommended role	Small or unsorted datasets	Ordered retrieval	Frequent exact-key lookups

Final recommendation: Use a hash table for frequent exact registration-number searches. Keep Linear Search as a simple baseline and Binary Search as a useful alternative when sorted retrieval and range queries are required. The recommendation should be confirmed by the experimental measurements collected in Tasks 8–10.

Task 12 – Day-4 Design Challenge: Re-evaluate and Adapt the Design

A. Impact Analysis

The revised operating conditions are:

- Database size: 1,000,000 student records.
- Search workload: 50,000 requests per hour.
- Record updates: New student records are added once per day.

The search rate is

$$\frac{50{,}000}{3{,}600} \approx 13.89 searches per second$$

Over 24 hours, the system would process up to

$$50,000 \times 24 = 1,200,000$$

search requests if the stated hourly workload continues throughout the day.

These requirements change the design priorities. The system must support frequent searches, predictable response times, and efficient preparation of the index. Since new records are added only once per day, the system can build or update its search structure in a scheduled process rather than repeatedly rebuilding it for every request.

The main assumption affected is that preprocessing cost is important relative to search cost. Under the new conditions, the database can be prepared once and then reused for a large number of searches.

B. Evaluation of the Original Design (2 Marks)

The original recommendation was Hash Table Search.

For one million records, the expected complexity is:

$$T_{build}(n) = \Theta(n)$$

$$E[T_{search}(n)] = \Theta(1)$$

For 50,000 searches, the expected total computational cost is

$$T_{total} = \Theta(n + q)$$

where $q = 50,000$ for one hour of searches.

The initial table construction processes one million records. Once built, the table can support all 50,000 searches without sorting or rebuilding the entire table for each request.

By comparison, Binary Search requires a one-time sorting cost of $\Theta(n \log n)$ and $\Theta(\log n)$ per search. Its total cost is

$$T_{binary} = \Theta(n \log n + q \log n)$$

Both algorithms are practical, but hashing has a lower expected computational cost for repeated exact-key searches.

Linear Search is unsuitable as the primary design because its worst-case total cost is $\Theta(qn)$. With one million records and 50,000 searches, this can require up to

$$50,000 \times 1,000,000 = 50,000,000,000$$

registration-number comparisons in one hour.

This is a theoretical upper bound on comparison count for the specified workload, not a measured execution-time result.

C. Adaptation or Redesign (2 Marks)

Decision: Retain and modify the original hash-table design.

The revised system should use a persistent database index or an in-memory hash index, depending on the available memory, durability requirements, and database platform.

Revised architecture

Student Records Database

1,000,000 existing records

Daily Index Preparation / Update

Build the index or insert newly added records

Registration Number Hash Index

Key → Student Record

Search Requests

50,000 requests per hour

What changes?

1. The index is prepared once and reused for subsequent requests.
2. Newly added records are inserted into the index during the daily update.
3. Registration numbers are kept unique using a uniqueness constraint.
4. The index is persisted or rebuilt safely after a restart.
5. Search performance is monitored under the actual workload.

If a relational database already supports an efficient index on the unique Registration Number column, its native indexing mechanism may be preferable to maintaining a separate custom hash table. A database's exact index implementation and performance depend on its platform and configuration.

Complexity after modification

If a hash table is rebuilt once per day:

$$T_{day} = \Theta(n + q)$$

in expected computational complexity, where n is the existing dataset size and q is the total number of searches during the day.

If only the new records are inserted, the daily update cost is expected to be proportional to the number of new records rather than the entire database, assuming suitable hash-table capacity and insertion behaviour.

D. Evidence-Based Defence (2 Marks)

The final design is defended using the following analysis.

Factor	Evidence and implication
Dataset size	One million records makes repeated full scans expensive
Search frequency	50,000 requests per hour favours efficient repeated lookup
Update frequency	Daily additions allow the search structure to be reused
Hash preprocessing	Expected $\Theta(n)$ initial construction cost
Hash lookup	Expected $\Theta(1)$ per query under suitable hashing assumptions
Memory	Additional memory is needed for the hash index
Reliability	Persistent storage, backups, and unique-key validation are required

For one million records and 50,000 hourly searches, the expected complexity of hashing is

$$\Theta(1,000,000 + 50,000)$$

per initial preparation plus one hour of lookups, ignoring constant factors.

The original Binary Search alternative has the total complexity

$$\Theta(n \log n + q \log n)$$

when sorting is needed. Hashing therefore remains attractive for this workload, especially because updates occur infrequently.

The final decision should also consider measured execution time, memory consumption, database durability, and collision behaviour. The supplied program can establish a basic benchmark, but the final report should include actual measured values from the execution environment rather than assumed timings.

Overall Conclusion

This assignment examined Linear Search, Recursive Binary Search, and Hash Table Search for a university student-record system. Mathematical analysis shows that Linear Search grows linearly, Binary Search grows logarithmically after sorting, and Hash Table Search provides expected constant-time lookups after expected linear-time construction.

For the revised requirement of one million records and 50,000 searches per hour, retaining and modifying the hash-based design is the most suitable choice for exact registration-number retrieval, subject to adequate memory and a reliable indexing implementation. The design can reuse its index between searches and incorporate daily record additions efficiently.

The final implementation should be validated using the same datasets, consistent test conditions, recorded execution times, comparison counts, and preprocessing costs. These results will provide the experimental evidence needed to support the theoretical recommendation.